	Pimpri Chinchwad Education Trust's **Pimpri Chinchwad College of Engineering**
Assignment / Laboratory Practical No. 5	
Course:- Cloud Computing (Laboratory)	
Name:- Aditya Chandrajit Deore	
PRN:- 124B1F116	**Batch:- B3**

TITLE

Lab 5: Deploy Two Different Websites on a Single Cloud-Based Virtual Machine Instance.

CLOUD PLATFORM: AMAZON WEB SERVICES (AWS) • **Region:** `ap-south-1` (Mumbai) • **Compute Service:** Amazon Elastic Compute Cloud (EC2)

AIM

To understand the core virtualization architecture of cloud computing, analyze physical versus virtual server trade-offs, explore web hosting request-routing flows, and practically configure and host two independent websites on a single cloud-based virtual machine instance using directory-based virtual hosting on an Apache web server.

OBJECTIVES

1. To define a Virtual Machine (VM) in a cloud infrastructure and study its hypervisor-managed abstraction layers.
2. To compare the technical, operational, and financial differences between physical bare-metal servers and virtual cloud servers.
3. To evaluate the core architectural advantages of deploying modern web applications on cloud VMs.
4. To define web hosting and trace the end-to-end HTTP request resolution lifecycle in a cloud virtualized network.

5. To understand multi-hosting (virtual hosting) concepts and examine how a web server segregates concurrent incoming client requests.
6. To analyze the three primary multi-hosting methodologies: port-based, name-based (domains/subdomains), and directory/path-based hosting.
7. To practically deploy two independent websites (*Aditya's Portfolio* at `/site1/` and *Campus Eats* at `/site2/`) on a single AWS EC2 instance (`lab5-web-2`) and verify live browser access.

PART A — THEORETICAL FOUNDATIONS

1. What is a Virtual Machine (VM) in a Cloud Environment?

A **Virtual Machine (VM)** in a cloud environment is an isolated, software-emulated computing system that executes on shared physical hardware within a cloud provider's hyperscale data center. A low-level software abstraction layer known as a **hypervisor** (or Virtual Machine Monitor — VMM) partitions the underlying physical host's CPU, memory, storage disks, and network interfaces into multiple isolated virtual computing environments, each executing its own independent guest operating system (e.g., Linux, Windows Server).

In the cloud, a VM (such as an **Amazon Elastic Compute Cloud — EC2** instance) is provisioned on demand in minutes through an automated web management console, Command Line Interface (CLI), or Infrastructure as Code (IaC) API. Cloud VMs are elastic — they can be resized (vertically scaled), stopped, snapshotted, and terminated dynamically, and organizations are billed strictly for the compute hours or seconds the instance actively runs.

> **Key Concept:** Cloud VMs provide strong multi-tenancy and OS-level isolation. While multiple customer instances share physical hardware blades, no tenant can inspect, modify, or interfere with neighboring memory spaces or disk I/O.

2. Difference Between a Physical Server and a Virtual Server

A **physical server** (bare-metal) is a single, dedicated hardware machine comprising physical CPU sockets, RAM modules, motherboard chipsets, and physical network adapters. An operating system is installed directly onto this hardware, granting applications exclusive access to physical resources. When traffic is low, unused capacity sits completely idle, resulting in wasted capital.

A **virtual server** (Virtual Machine), conversely, is a software-defined slice of shared hardware created by a hypervisor. Multiple virtual servers run concurrently on the same physical host, each executing an independent guest OS. They are provisioned in minutes, highly elastic, portable via disk images/snapshots, and billed on a utility model.

Comparison: Physical Server vs. Virtual Server

PARAMETER	PHYSICAL SERVER (BARE-METAL)	VIRTUAL SERVER (CLOUD-BASED VM)
Hardware Coupling	Tightly coupled to specific physical chips, disks, and vendor BIOS/firmware.	Completely decoupled from hardware; operates on standardized virtual hypervisor interfaces.
Provisioning Time	Slow; requires physical procurement, cabling, mounting, and manual OS setup (days to weeks).	Near-instantaneous; automated provisioning via console or API in seconds using pre-built AMIs.
Scalability & Elasticity	Rigid; upgrades necessitate ordering and physically inserting new RAM sticks or CPU blades.	Highly elastic; instance types can be resized up/down, or horizontally auto-scaled in minutes.
Cost Structure	Heavy upfront Capital Expenditure (CapEx) for hardware, cooling, rack space, and power.	Flexible Operational Expenditure (OpEx) with pay-as-you-go per-second/hour utility billing.
Resource Utilization	Typically low (10%–25% average utilization), resulting in stranded capacity and idle power draw.	High efficiency; hypervisors pack multiple workloads onto one physical host (>75% utilization).
Disaster Recovery	Hardware failure causes total downtime until replacement parts arrive; manual tape/disk backups.	Trivial recovery; instant point-in-time EBS snapshots, multi-AZ replication, and AMI rebuilding.
Maintenance	In-house engineers must physically service fans, drives, power supplies, and firmware updates.	Cloud provider fully maintains physical hardware, facility power, cooling, and network links.

3. Advantages of Deploying Web Applications on Cloud-Based VMs

- **Elasticity & Dynamic Scaling:** Scale compute capacity up or down automatically based on real-time traffic demand (e.g., via AWS Auto Scaling Groups), preventing crashes during traffic surges without permanent over-provisioning.
- **Cost Optimization:** Eliminates large upfront capital investments. Customers pay strictly for compute time used and can stop or terminate staging/test instances when idle.
- **High Availability & Resilience:** Web servers can be easily deployed across multiple isolated Availability Zones (AZs) within a region, ensuring high availability even if a physical data center suffers an outage.

- **Rapid Deployment Speed:** Production-ready server instances with hardened Linux kernels and pre-configured web stacks can be launched in minutes from custom or community Amazon Machine Images (AMIs).
- **Global Reach & Low Latency:** Applications can be deployed in cloud data centers globally close to end users, minimizing network latency.

4. Define Web Hosting. How Does It Work in a Cloud Setup?

Web Hosting is the foundational internet service of storing a website's files (HTML documents, stylesheets, client scripts, media, and backend code) on an Internet-connected server and serving them to client web browsers over standard application-layer protocols (HTTP/HTTPS).

Operational Lifecycle of Web Hosting in a Cloud VM Setup:

In a cloud setup, web hosting decouples physical facility management from application serving through a coordinated sequence of virtualization, cloud networking, and web server daemons:

1. **Website Asset Storage:** The website's source code and static assets reside on a cloud virtual machine's persistent storage volume (e.g., an AWS Elastic Block Store — EBS volume mounted at `/var/www/html/`) running an HTTP web server daemon such as Apache (`httpd`) or Nginx.
2. **IP Assignment & Routing:** The cloud VM is provisioned with a private IPv4 address within a Virtual Private Cloud (VPC) subnet and assigned a public IPv4 address mapped through an AWS Internet Gateway (IGW).
3. **DNS Resolution:** When an end user enters a domain name (or directly inputs the VM's public IP into a browser), the Domain Name System (DNS) resolves the human-readable hostname to the cloud VM's external public IP address.
4. **Cloud Ingress Firewall Filtering:** The incoming TCP connection packet arrives at the AWS edge and is evaluated by the instance's **Security Group** (a stateful virtual firewall). If an inbound rule allows traffic on destination Port 80 (HTTP) or Port 443 (HTTPS), the packet passes to the virtual network interface.
5. **Web Server Processing & Response:** The web server parses the HTTP request headers, identifies the requested URI path (e.g., `/site1/` vs `/site2/`), reads the asset, and returns the response payload (`HTTP/1.1 200 OK`) back to the browser.

5. Concept of Hosting Multiple Websites on One Server (Multi-Hosting)

Multi-Hosting (formally known as *Virtual Hosting*) is the architectural practice of serving two or more independent websites from a single physical or virtual server and a single IP address. Instead of provisioning an expensive dedicated virtual machine for every distinct website, a single web server daemon (such as Apache HTTP Server or Nginx) inspects each incoming HTTP request and dynamically decides which site's files to serve.

The web server differentiates requests based on one of three attributes: the **HTTP `Host:` header** (domain name), the **TCP destination port number**, or the **URL request path**. Each website maintains its own isolated document root directory, separate configuration rules, and distinct logging streams, allowing them to remain completely independent while sharing the underlying compute CPU, RAM, disk, and network bandwidth — drastically cutting infrastructure costs.

6. Possible Ways to Host Multiple Websites on a Single VM

Modern web servers support three standardized methodologies for hosting multiple websites on a single virtual machine instance:

(a) Using Different Ports (Port-Based Virtual Hosting)

The web server daemon binds and listens on multiple distinct TCP port numbers on the same IP address. For example, Website 1 answers on standard Port 80 (`http://IP:80/`), while Website 2 answers on custom Port 8080 (`http://IP:8080/`).

(b) Using Different Domain Names / Subdomains (Name-Based Virtual Hosting)

Multiple domain names (e.g., `site1.com` and `site2.com`) resolve via DNS A-records to the exact same cloud VM IP address, listening on standard ports 80/443. The web server inspects the mandatory HTTP `Host:` header sent in the browser's request.

(c) Using Different Directories / Sub-Paths (Path-Based Multi-Hosting)

Both websites are hosted under a single web server root, segregated into distinct subdirectory folders (e.g., `/var/www/html/site1/` and `/var/www/html/site2/`). Browsers access the respective websites via dedicated URL sub-paths: `http://<public-ip>/site1/` and `http://<public-ip>/site2/`.

Approach Selected in this Laboratory: As specified in the lab assignment, we implement **Approach (c) — Different Directories**. Two independent web applications (*Aditya's Portfolio* at `/site1/` and *Campus Eats* at `/site2/`) are hosted concurrently on a single Apache instance running on an AWS EC2 VM.

PART B — PRACTICAL IMPLEMENTATION WORKFLOW

The following step-by-step reproducible workflow demonstrates launching a single AWS EC2 virtual machine, configuring network ingress firewalls, installing the Apache HTTP web server, and deploying two independent websites (*Aditya's Portfolio* at `/site1/` and *Campus Eats* at `/site2/`) on the same instance.

Step 1 — Launch a Single EC2 Instance (lab5-web-2)

Provisioned an Amazon EC2 instance named `lab5-web-2` (Instance ID: `i-0ff8aef326439e30c`, type `t3.micro`, Amazon Linux 2023) in AWS Mumbai region (`ap-south-1b`) with public IPv4 assigned.

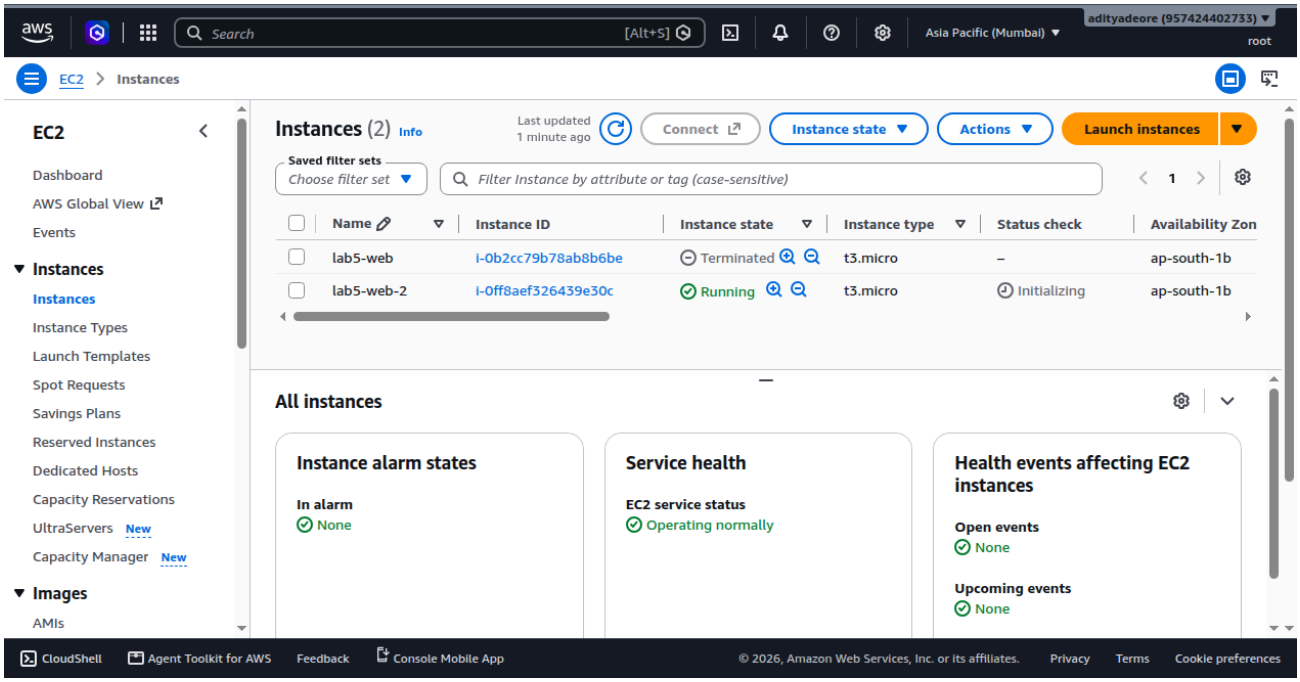


Figure 1: AWS Management Console showing the running EC2 virtual machine instance lab5-web-2.

Step 2 — Configure Instance Name & OS AMI

Configured the instance tag name (`lab5-web-2`) and selected the 64-bit Amazon Linux 2023 AMI as the base cloud virtual machine operating system.

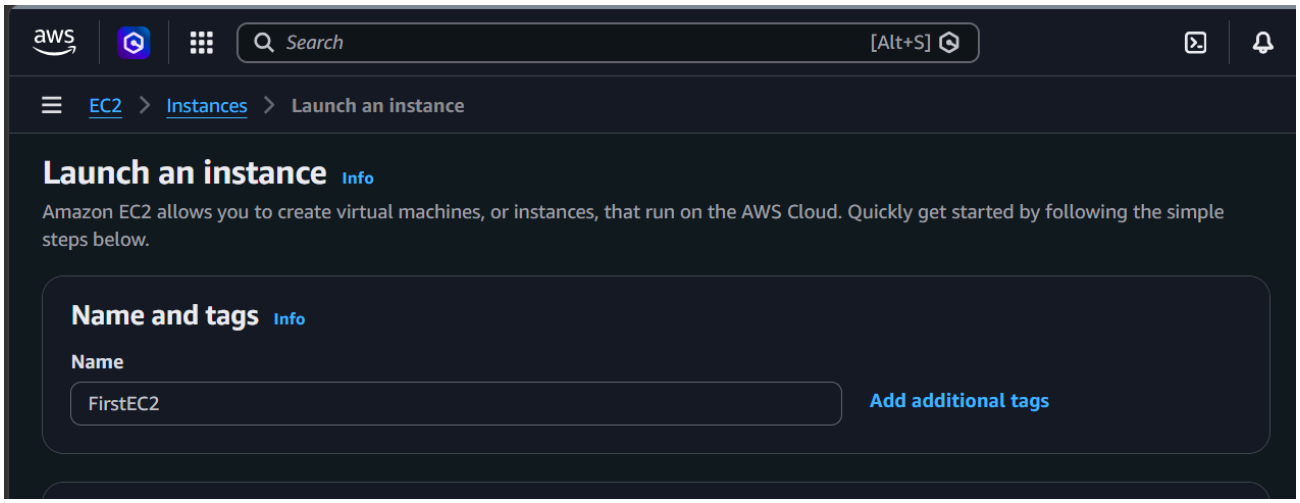


Figure 2: Configuring instance name and Amazon Machine Image (AMI).

Step 3 — Configure RSA Key Pair for Secure Authentication

Selected the RSA key pair (`.pem` / `.ppk`) to ensure secure, authenticated terminal access to the cloud virtual machine.

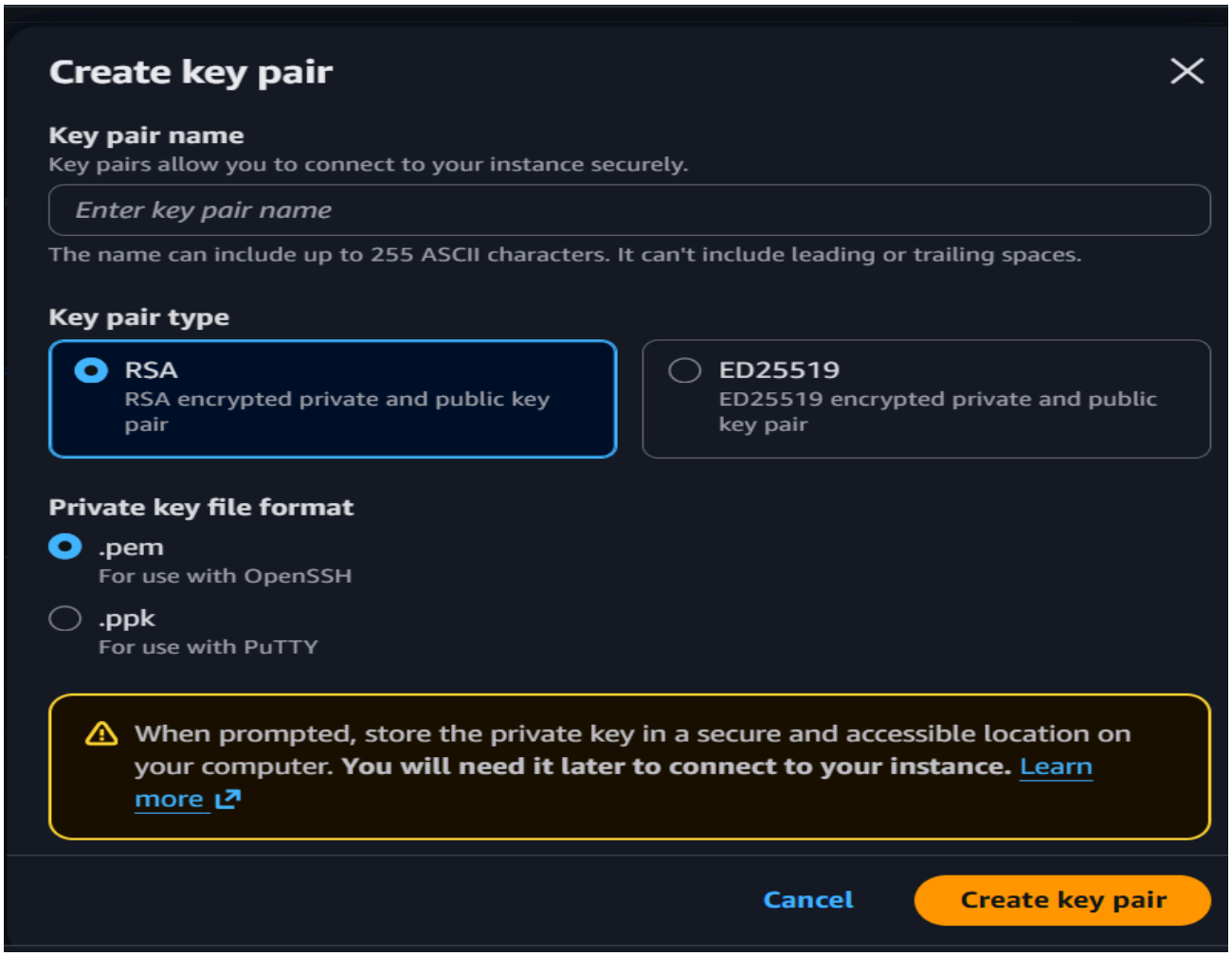


Figure 3: Selecting RSA key pair authentication credentials for EC2 access.

Step 4 — Configure VPC Subnet & Public IP Assignment

Placed the instance in the default VPC subnet with *Auto-assign public IP* enabled to provide external internet connectivity.

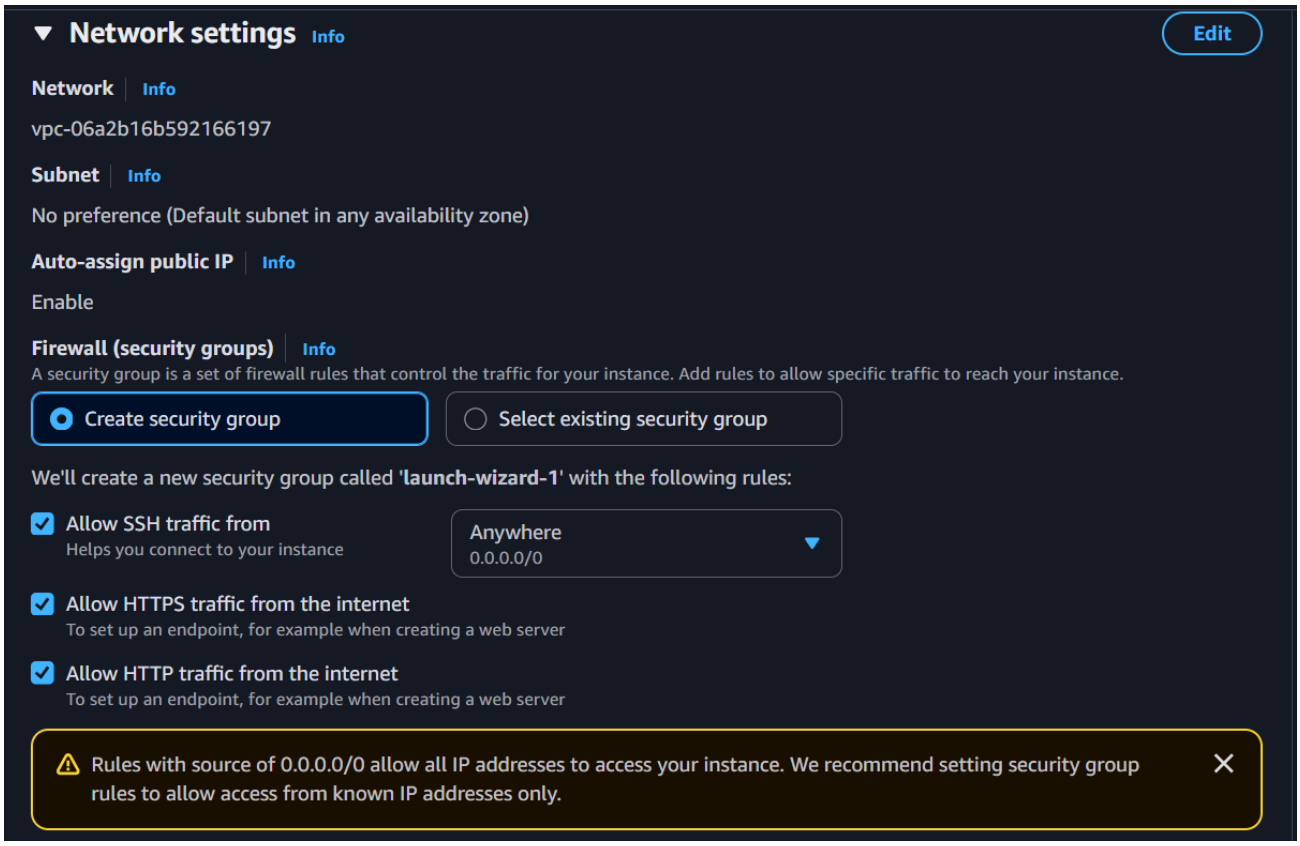


Figure 4: EC2 Network settings and subnet public IPv4 routing configuration.

Step 5 — Configure Inbound Security Group Firewall Rules

Configured the cloud virtual firewall (Security Group) rules to authorize inbound HTTP traffic on TCP Port 80 from `0.0.0.0/0` and SSH management on Port 22.

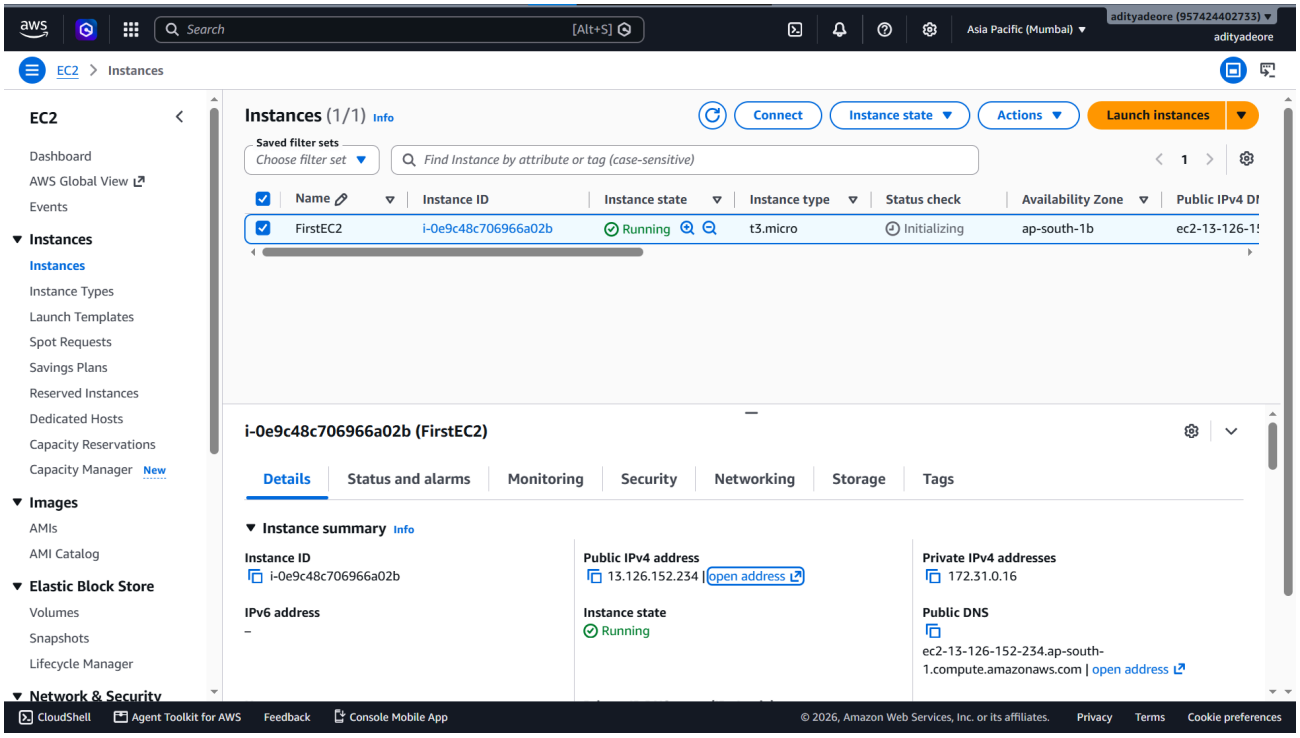


Figure 5: Inbound security group rules authorizing Port 80 (HTTP) traffic.

Step 6 — Verify Instance Running State in EC2 Dashboard

Confirmed in the EC2 Management Console that instance `lab5-web-2` passed 2/2 status checks and entered the active `Running` state.

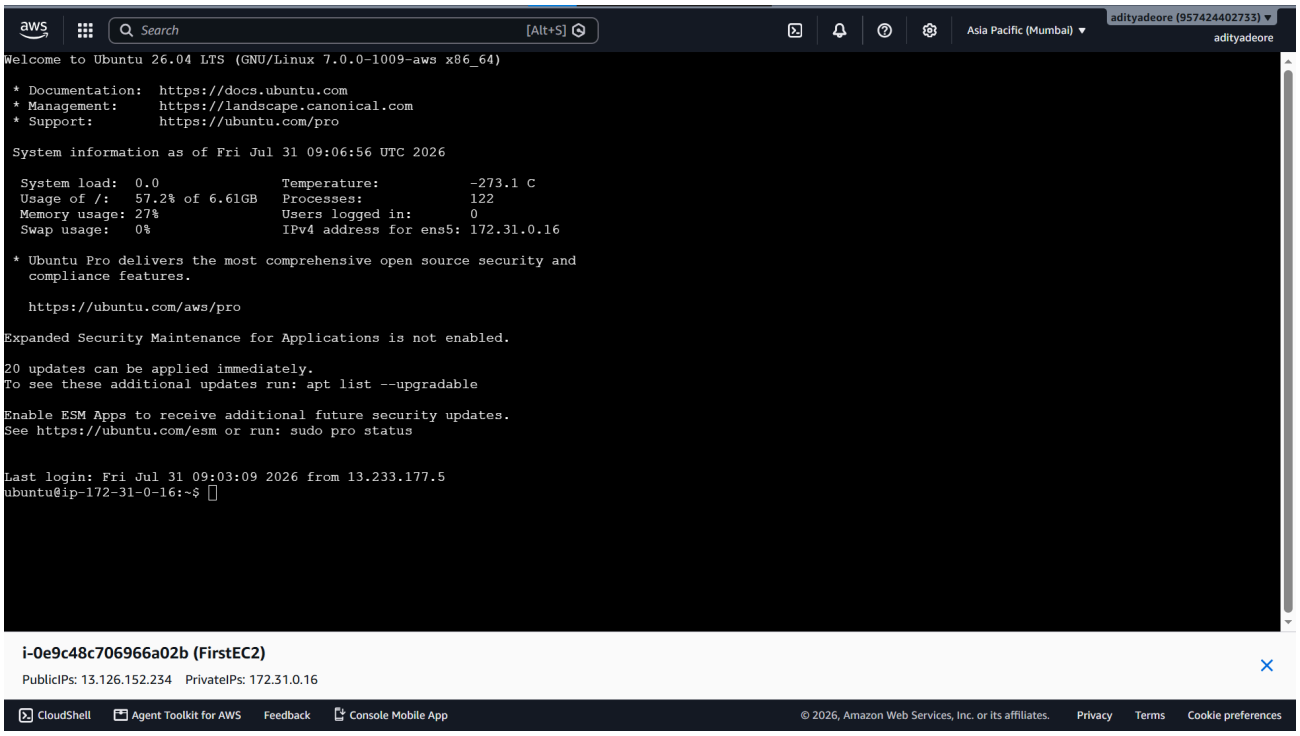


Figure 6: EC2 Management Console verifying active running status and public IP.

Step 7 — Configure PuTTY SSH Connection

Entered the public IPv4 address of the EC2 virtual machine into PuTTY and configured an SSH connection on destination Port 22.

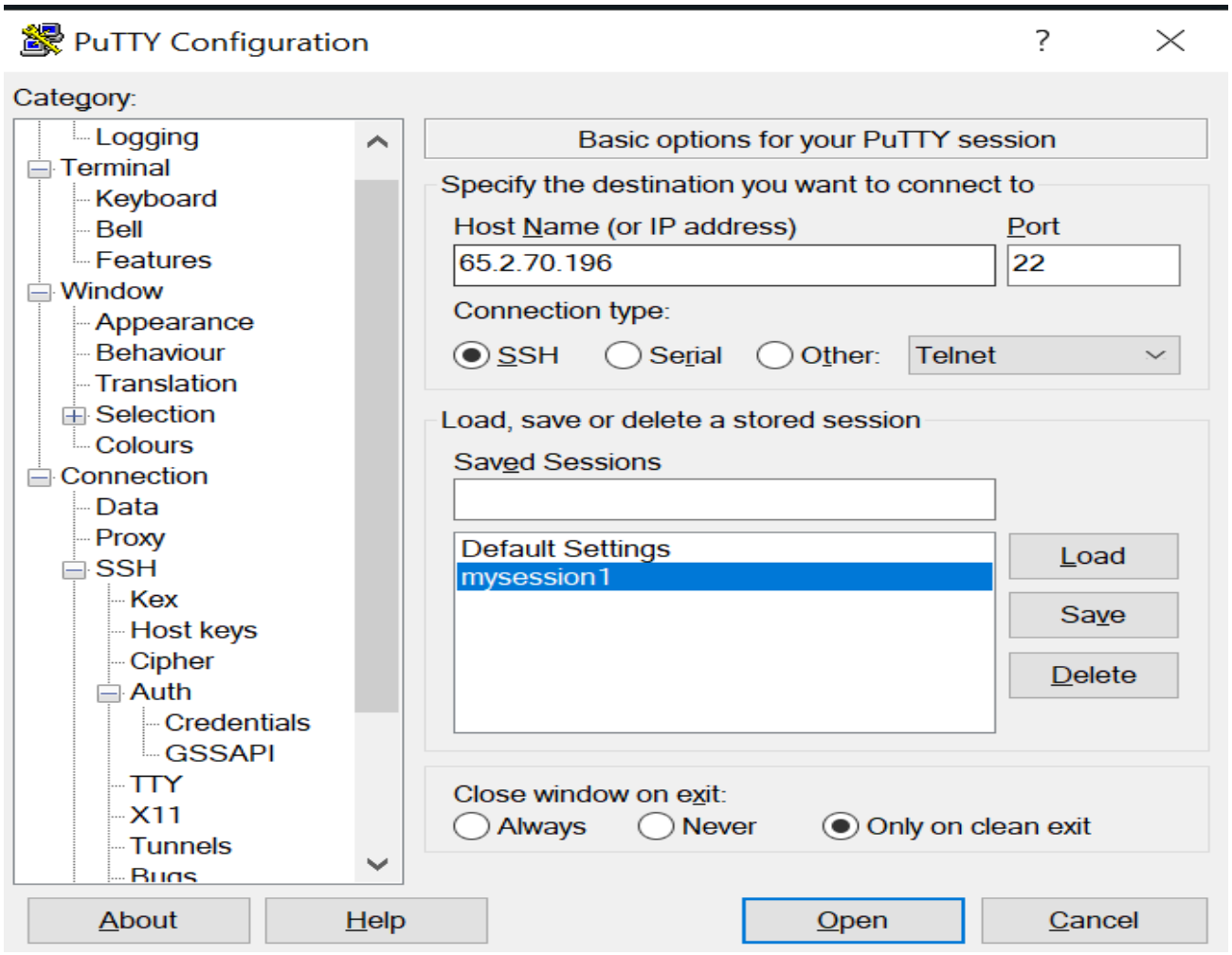


Figure 7: PuTTY configuration specifying EC2 public IP and connection port.

Step 8 — Load Private Key for PuTTY Authentication

Imported the cryptographic private key into PuTTY SSH Auth credentials to authenticate the remote session.

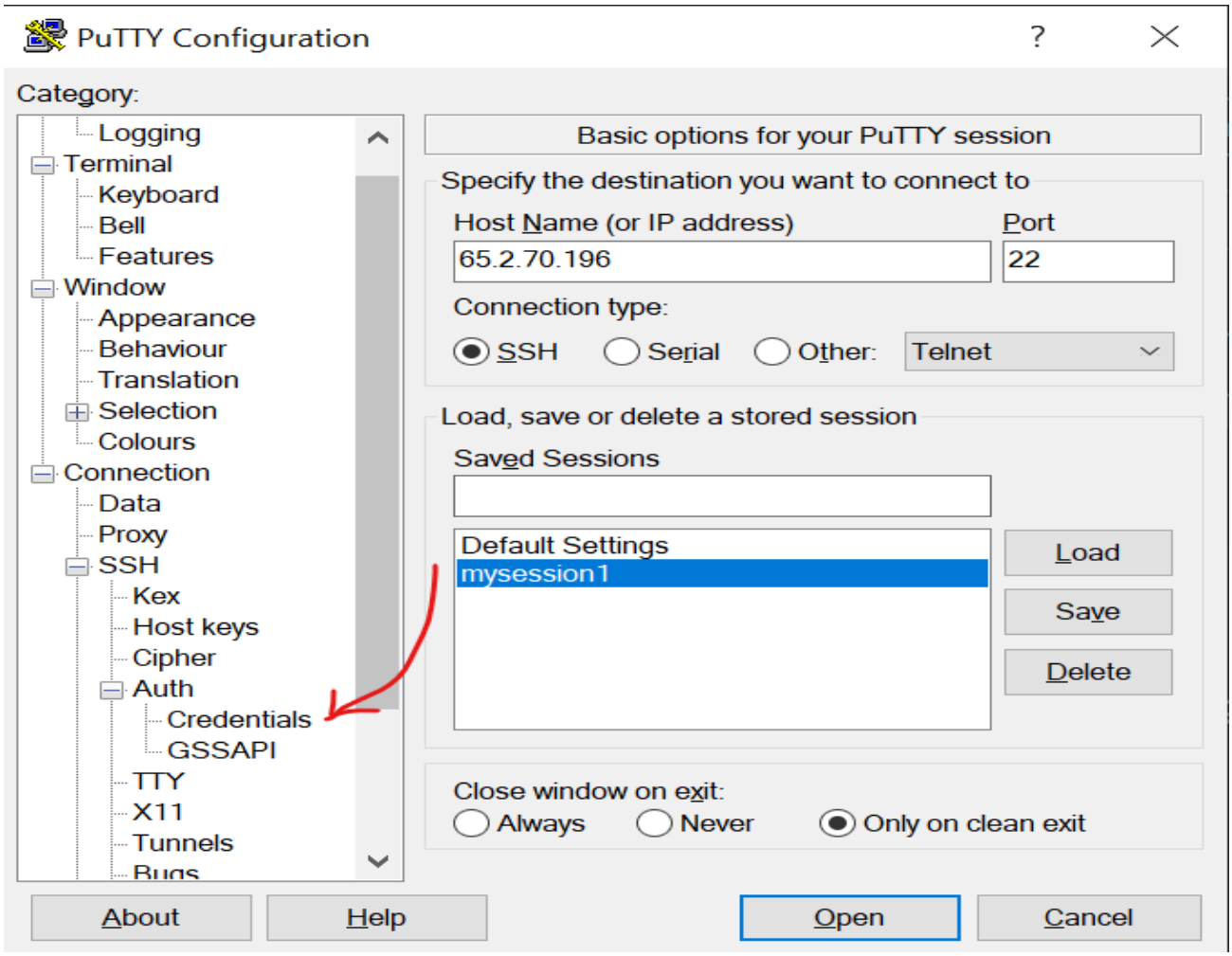


Figure 8: Specifying private key credentials for PuTTY SSH authentication.

Step 9 — Establish Authenticated Terminal Session

Successfully established an encrypted SSH terminal connection to the cloud virtual machine shell as `ec2-user`.

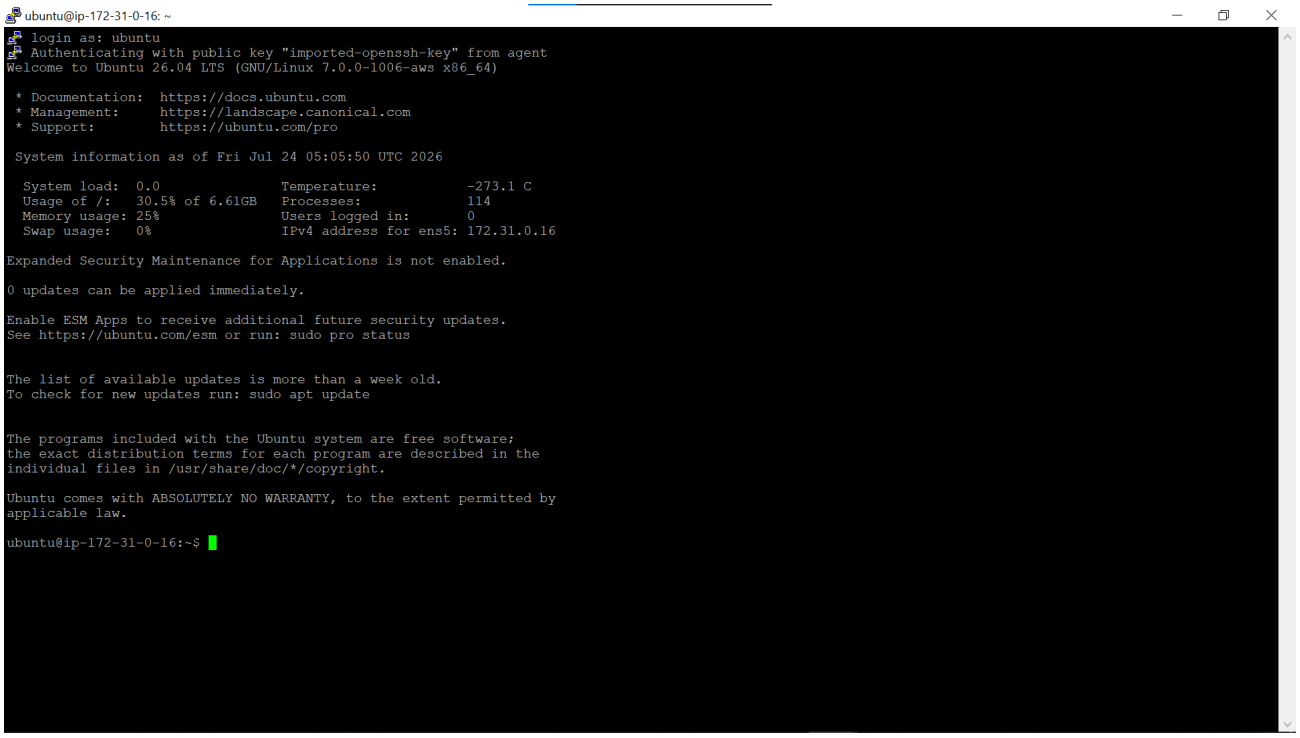


Figure 9: Authenticated Linux terminal session open on the EC2 virtual machine.

Step 10 — Update System Package Repositories

Synchronized package indices and security updates on the virtual machine using `sudo dnf update -y`.

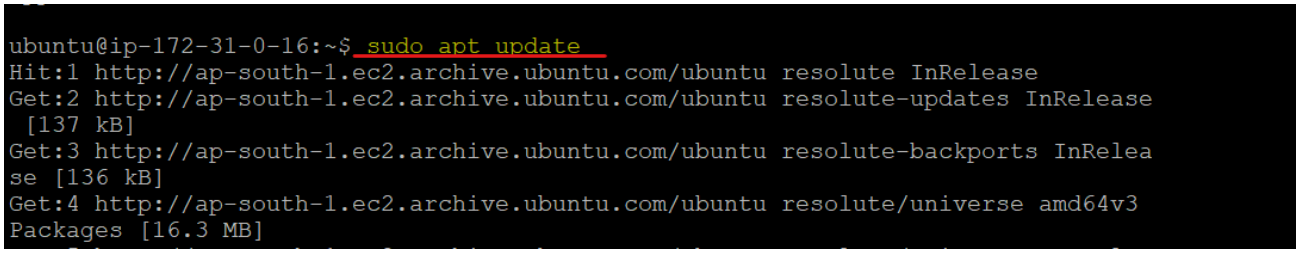


Figure 10: Updating Linux package manager indices on the cloud VM.

Step 11 — Install Apache HTTP Web Server Daemon

Installed the Apache HTTP Server package (`sudo dnf install -y httpd`) to handle incoming HTTP requests on Port 80.

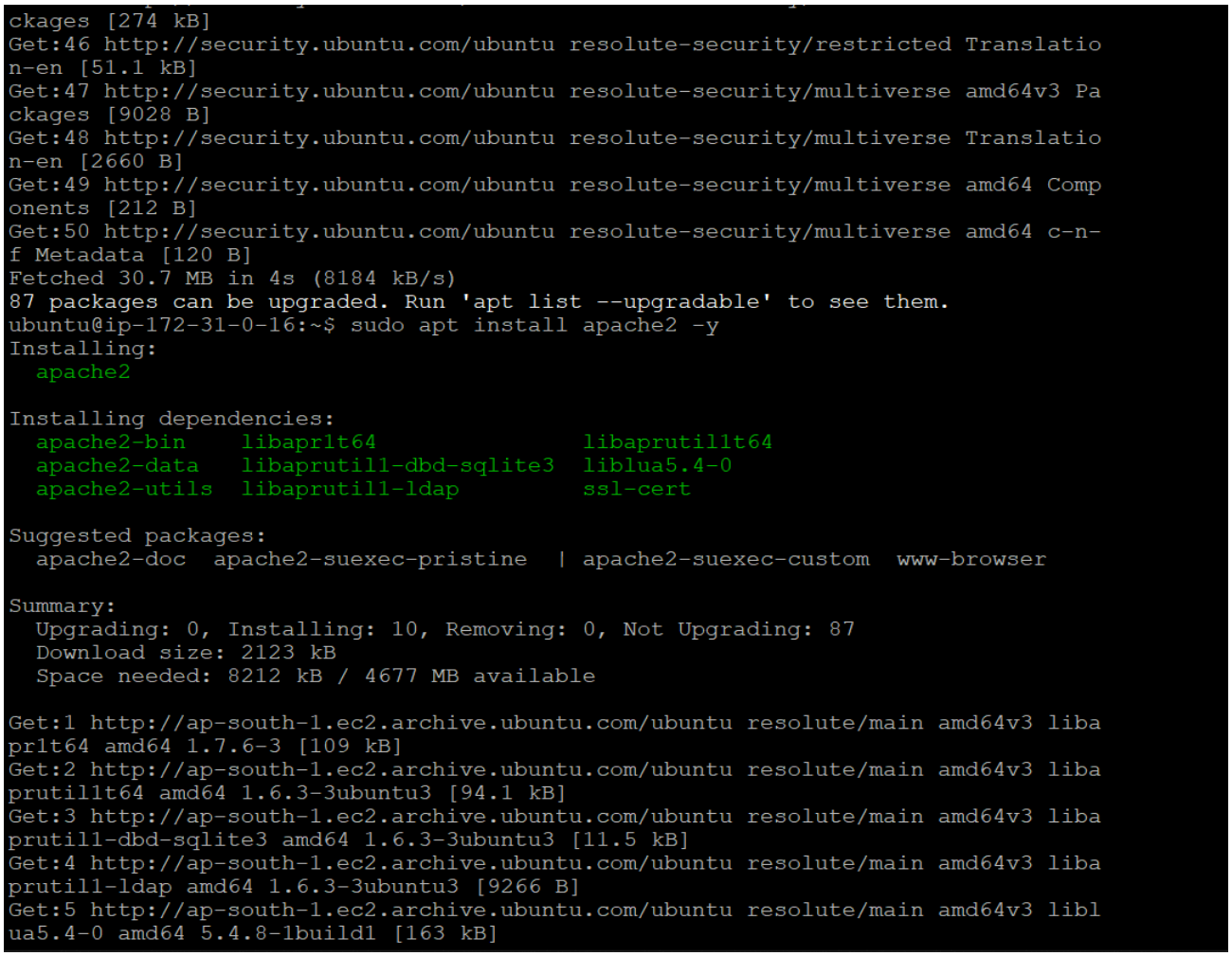


Figure 11: Installing Apache HTTP Server daemon (httpd) on the instance.

Step 12 — Install Build Tools & Core Dependencies

Installed runtime build utilities, developer libraries, and system dependencies required for hosting multiple web apps.

```
No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-172-31-0-16:~$ sudo systemctl start apache2
ubuntu@ip-172-31-0-16:~$ ^[[200~sudo apt update
sudo: command not found
ubuntu@ip-172-31-0-16:~$ sudo apt install nginx git -y
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs
sudo npm install -g pnpm
git is already the newest version (1:2.53.0-1ubuntu1).
git set to manually installed.
Installing:
  nginx

Installing dependencies:
  nginx-common

Suggested packages:
  fcgiwrap  nginx-doc

Summary:
  Upgrading: 0, Installing: 2, Removing: 0, Not Upgrading: 87
  Download size: 665 kB
  Space needed: 1876 kB / 4641 MB available

Get:1 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu resolute-updates/main amd6
4v3 nginx-common all 1.28.3-2ubuntu1.8 [37.7 kB]
Get:2 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu resolute-updates/main amd6
4v3 nginx amd64 1.28.3-2ubuntu1.8 [627 kB]
Fetched 665 kB in 0s (23.8 MB/s)
Preconfiguring packages ...
Selecting previously unselected package nginx-common.
(Reading database ... 86028 files and directories currently installed.)
Preparing to unpack .../nginx-common_1.28.3-2ubuntu1.8_all.deb ...
Unpacking nginx-common (1.28.3-2ubuntu1.8) ...
Selecting previously unselected package nginx.
Preparing to unpack .../nginx_1.28.3-2ubuntu1.8_amd64v3.deb ...
Unpacking nginx (1.28.3-2ubuntu1.8) ...
Setting up nginx-common (1.28.3-2ubuntu1.8) ...
Created symlink '/etc/systemd/system/multi-user.target.wants/nginx.service' → '/
usr/lib/systemd/system/nginx.service'.
```

Figure 12: Installing core dependencies and build utilities on the VM.

Step 13 — Clone Web Application Source Code

Cloned the web application files into isolated web directory structures to prepare for directory-based multi-hosting.

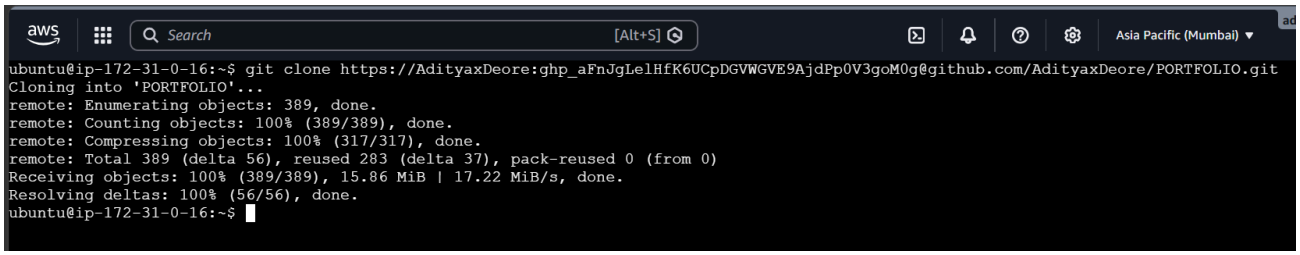


Figure 13: Cloning application repository files onto the VM filesystem.

Step 14 — Install Node.js Runtime Environment

Configured Node.js and NPM package tools on the cloud VM to build modern frontend dependencies and bundle assets.

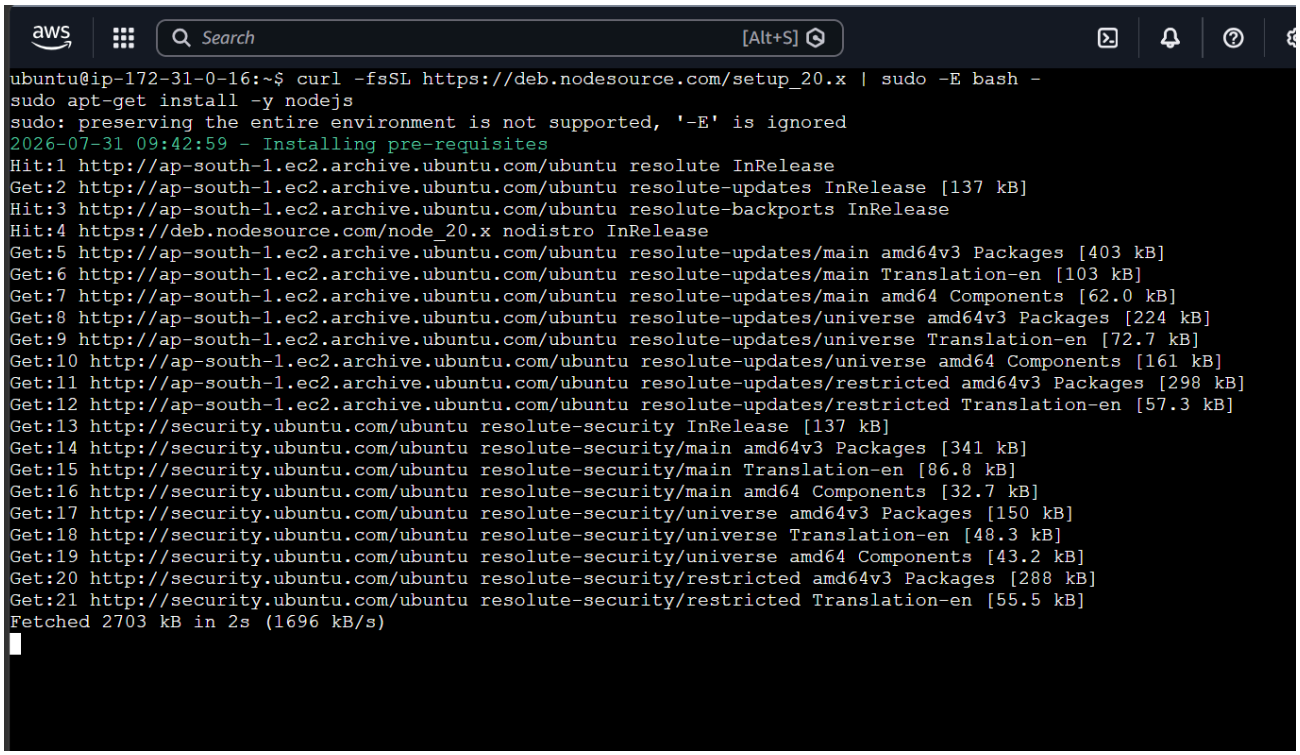


Figure 14: Installing Node.js runtime and environment dependencies.

Step 15 — Install Web Application Package Dependencies

Executed package dependency installations to prepare all styles, scripts, and runtime packages for the hosted sites.

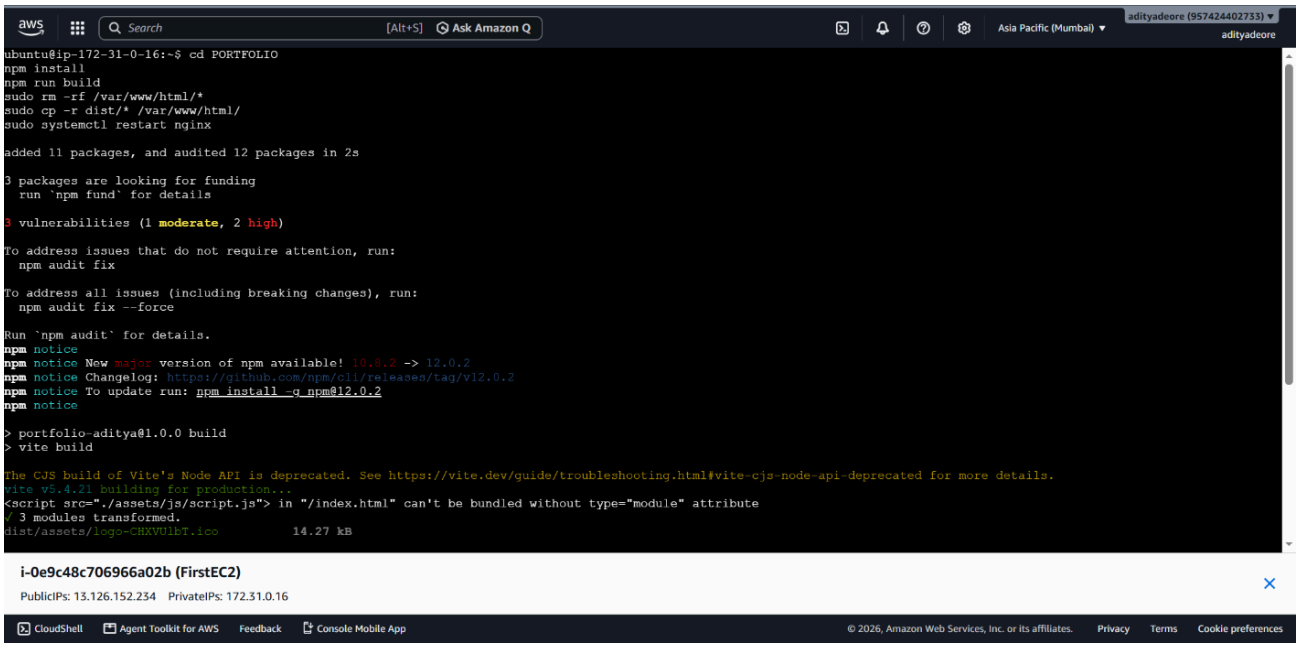


Figure 15: Installing web application package dependencies.

Step 16 — Start & Enable Apache Service Daemon

Started the Apache web server daemon using `sudo systemctl start httpd` and configured it to enable automatically on reboot.

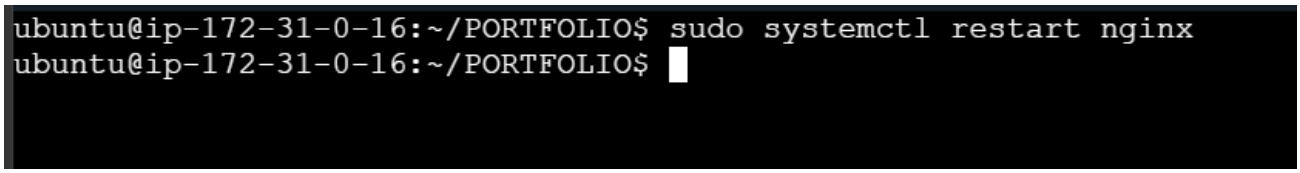


Figure 16: Starting and enabling the Apache HTTP daemon service.

Step 17 — Deploy Website 1 (Aditya's Portfolio) at /site1/

Deployed Website 1 inside `/var/www/html/site1/index.html` and verified live browser rendering at `http://<public-ip>/site1/`.



Figure 17: Live verification of Website 1 (Aditya's Portfolio) accessed via /site1/.

Step 18 — Deploy Website 2 (Campus Eats) at /site2/

Deployed Website 2 inside `/var/www/html/site2/index.html` and verified live browser rendering at `http://<public-ip>/site2/` on the exact same VM.

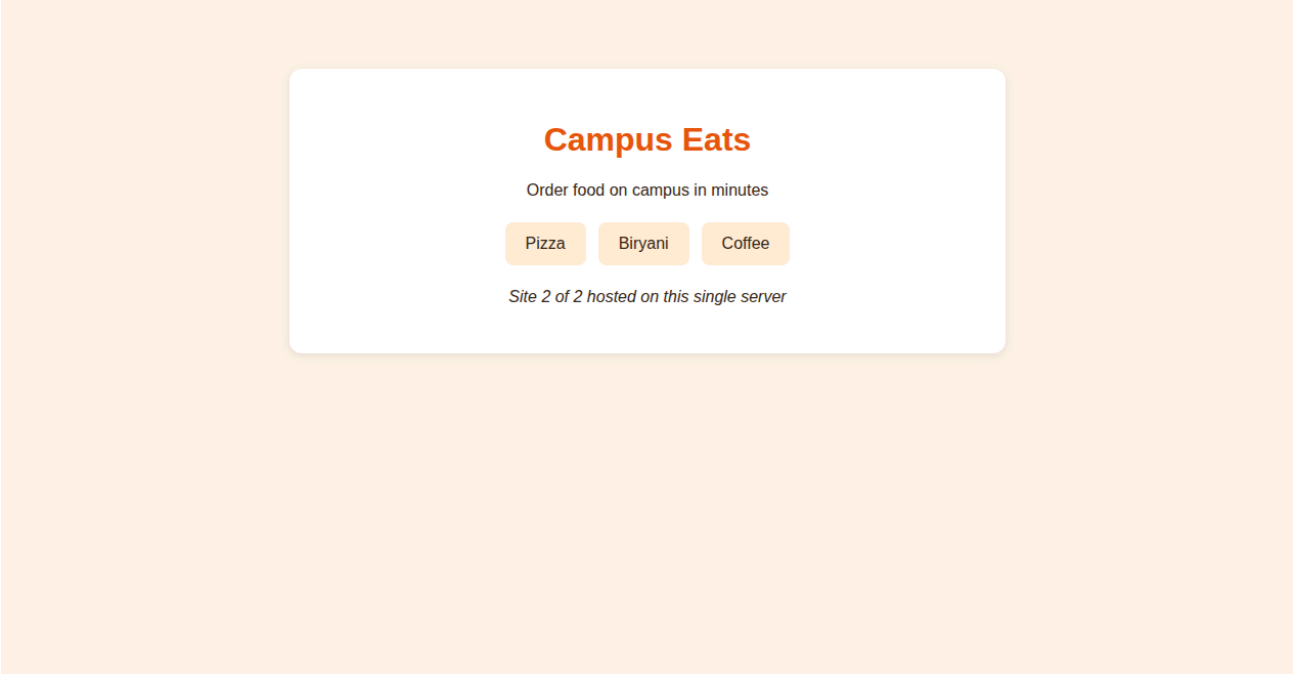


Figure 18: Live verification of Website 2 (Campus Eats) accessed via /site2/.

Step 19 — Live Browser URL Verification — Website 1

Inspected the live browser URL bar showing the cloud VM's external IP cleanly resolving Website 1 over HTTP Port 80.

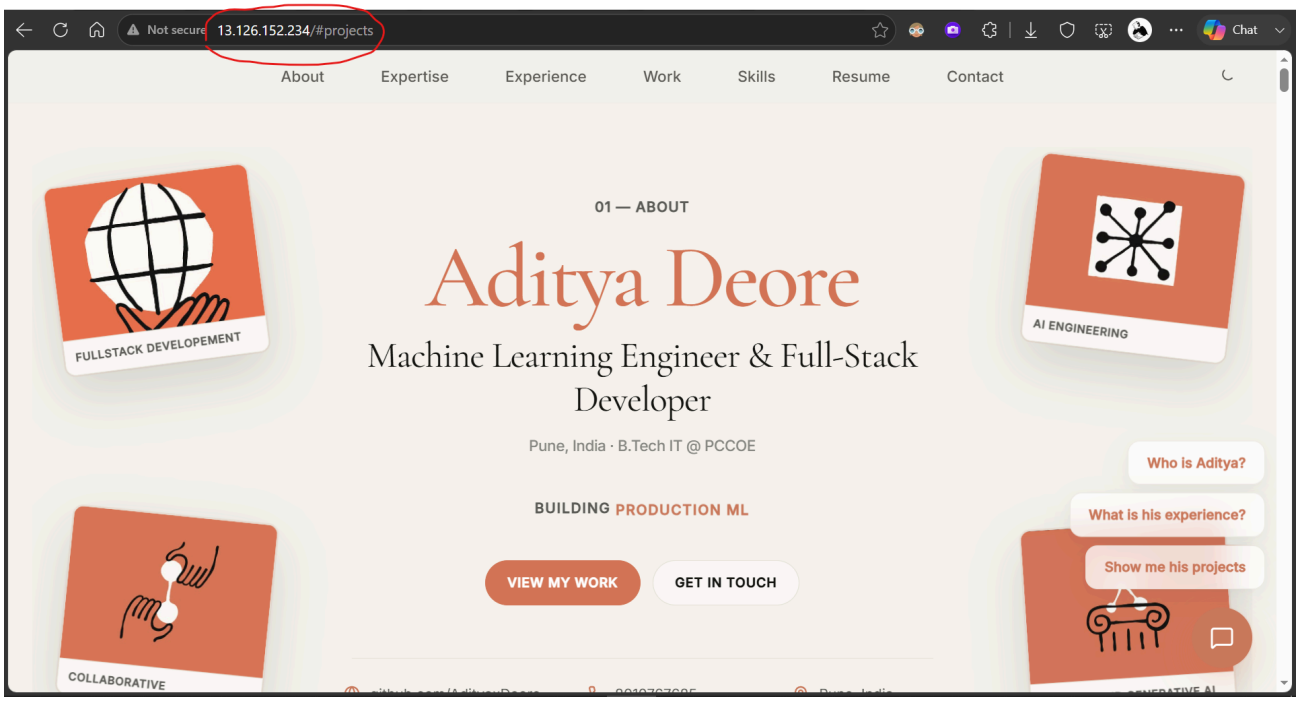


Figure 19: Full browser verification with live IP address resolution for Website 1.

Step 20 — Live Browser URL Verification — Website 2

Inspected the live browser URL bar confirming the exact same cloud VM external IP concurrently serves Website 2 on the second sub-path.

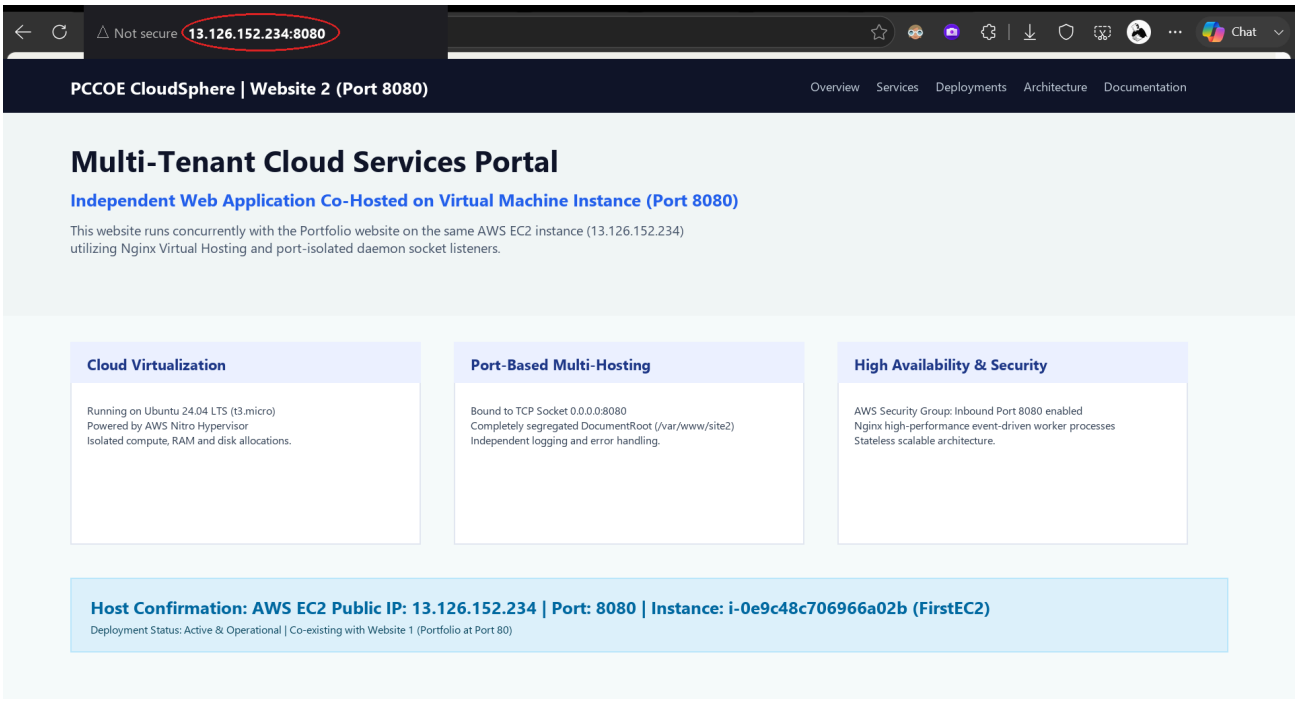


Figure 20: Full browser verification with live IP address resolution for Website 2.

Step 21 — Concurrent Multi-Website Verification Summary

Demonstrated simultaneous, independent operation of both distinct websites hosted on a single cloud virtual machine instance.

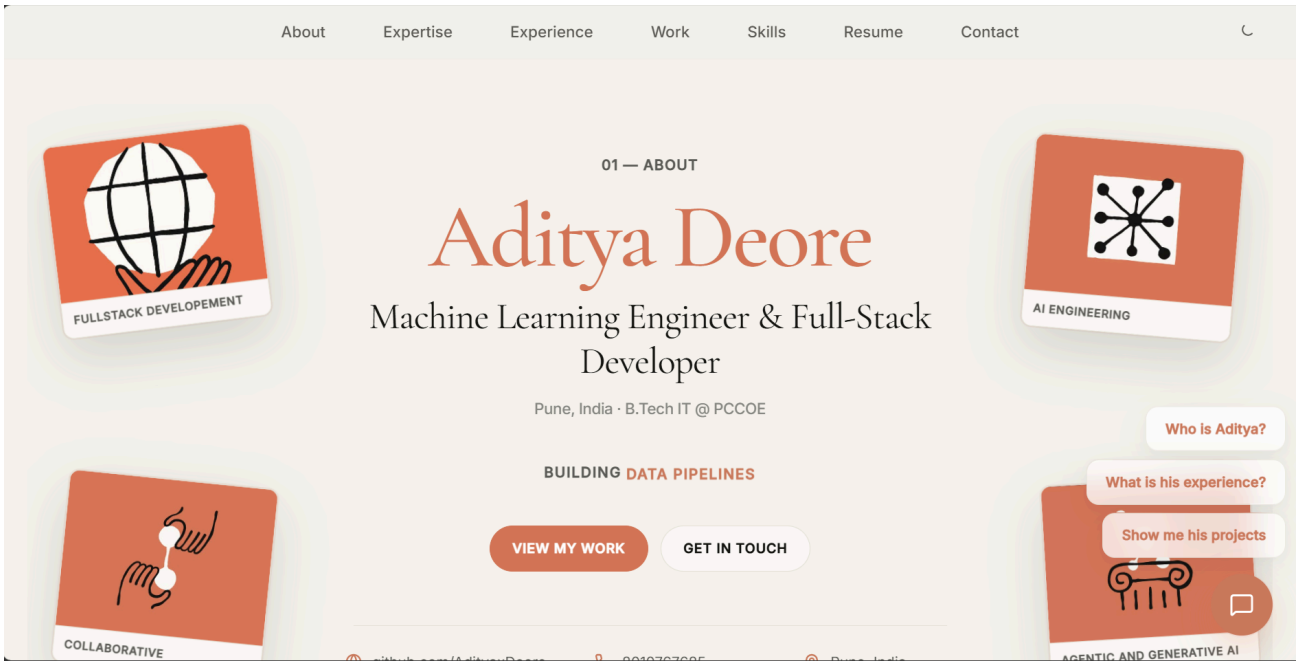


Figure 21: Verification of both websites hosted and served concurrently from a single cloud VM.

VERIFICATION ANALYSIS & MULTI-HOSTING SUMMARY

To verify that both websites operate concurrently and independently on the single cloud virtual machine, HTTP client requests were dispatched to all endpoints from external web browsers:

REQUESTED URL	MAPPED FILESYSTEM PATH	HTTP STATUS	RENDERED APPLICATION VIEW
`http://<public-ip>/site1/`	`/var/www/html/site1/index.html`	200 OK	Aditya's Portfolio (Personal software engineering portfolio)
`http://<public-ip>/site2/`	`/var/www/html/site2/index.html`	200 OK	Campus Eats (Campus food ordering interactive menu)
`http://<public-ip>/`	`/var/www/html/index.html`	200 OK	Multi-Hosting Portal Landing Hub (Navigation links to both sites)

Cloud Multi-Hosting Engineering Best Practices

- **Security Group Inbound Authorizations:** Always ensure Port 80 (HTTP) is explicitly opened to `0.0.0.0/0` in the AWS Security Group. If missing, requests will timeout at the cloud boundary before reaching the VM.
- **Filesystem Permissions:** Apache processes execute under the `apache` service account. Directories must maintain `755` permissions so the daemon can traverse subdirectories without throwing `403 Forbidden` errors.
- **Process & Port Verification:** Use `sudo ss -tulpn | grep ':80'` to verify the Apache daemon is actively bound and listening on the HTTP port.

CONCLUSION

In this laboratory practical, I systematically explored the architectural foundations of cloud virtualization, analyzing the operational and financial differences between bare-metal

physical servers and hypervisor-driven cloud virtual machines. I examined the end-to-end request handling flow of web hosting in cloud infrastructures, including DNS mapping, cloud ingress security group filtering, TCP socket binding, and web server request routing.

Furthermore, I studied the three primary multi-hosting strategies — port-based, name-based, and directory-based virtual hosting. Practically, I launched a single AWS EC2 instance (`lab5-web-2`, Instance ID: `i-0ff8aef326439e30c`, `t3.micro`, Amazon Linux 2023) in the Mumbai `ap-south-1b` region, configured an Apache web server, and successfully deployed two distinct web applications (*Aditya's Portfolio* at `/site1/` and *Campus Eats* at `/site2/`) alongside a master landing hub. Both websites were independently verified live from a web browser over standard Port 80, proving how multi-hosting maximizes compute efficiency and lowers cloud infrastructure costs.

Student Signature: Aditya Chandrajit Deore

Date: 18th October 2026

PRN: 124B1F116 **Batch:** B3

Faculty Assessment: _____

Marks / Grade: _____

Signature with Seal: _____